2.4 Preprocessing

The EDA shows that the images already share a fixed resolution and class mapping, but differ in brightness, coverage, and spatial appearance. Our preprocessing therefore keeps the original image content and applies a common numerical transformation before arranging the pixels for each model. The pipeline has four steps: convert the raw image to a floating-point tensor, scale its intensities, standardize it using training statistics, and construct the required input layout. The same fitted transformation is used for training, validation, and test images.

2.4.1 Tensor Conversion and Intensity Scaling

Let $X_i \in \{0, \ldots, 255\}^{28\times 28}$ be a raw image. The loader uses `ToTensor` to add a channel dimension, convert the unsigned 8-bit values to `float32`, and divide the intensities by 255:

$$U_{i,0,r,c} = \frac{X_{i,r,c}}{255}, \qquad U_i \in [0,1]^{1\times 28\times 28}. \tag{4}$$

Here, $r, c \in \{0, \ldots, 27\}$ denote the row and column, and the channel index is zero because the image is grayscale. For example, raw intensities 0, 128, and 255 become approximately 0, 0.5020, and 1. This conversion changes the numeric range while preserving the 784 pixels, their order, and the supplied label. Native resolution is retained; no interpolation or resizing is needed. Scaling takes place on the CPU in the dataset loader, before a batch is transferred to the selected training device.

2.4.2 Standardization Using Training Statistics

Scaling to $[0, 1]$ and standardization serve different purposes. Scaling sets a common numerical range; standardization centers that range relative to the training distribution and expresses variation in units of its standard deviation. We use one mean and one population standard deviation for the single grayscale channel, computed across all training images and positions. With $\mathcal{I}_{\text{tr}}$ denoting the frozen set of 54,000 training indices and $N = 54{,}000 \times 784 = 42{,}336{,}000$, the statistics are

$$\mu = \frac{1}{N} \sum_{i \in \mathcal{I}_{\text{tr}}} \sum_{r=0}^{27} \sum_{c=0}^{27} U_{i,0,r,c}, \qquad \sigma = \sqrt{\frac{1}{N} \sum_{i \in \mathcal{I}_{\text{tr}}} \sum_{r=0}^{27} \sum_{c=0}^{27} (U_{i,0,r,c} - \mu)^2}. \tag{5}$$

The measured values are $\mu = 0.286209$ and $\sigma = 0.353160$, rounded here to six decimal places. The implementation retains the full precision stored in the split cache. Each scaled image is transformed as

$$Z_{i,0,r,c} = \frac{U_{i,0,r,c} - \mu}{\sigma}. \tag{6}$$

These are global training-pixel statistics, rather than separate statistics for each image, class, or batch. Applying the same affine transformation to every image preserves differences in brightness between examples. Computing separate statistics for each class would also require the unknown target label when preprocessing a new image, so it is not part of this pipeline.

Table 5 summarizes the three numerical stages. After standardization, raw zero-valued background pixels become approximately -0.810, and raw values of 255 become approximately 2.021. Negative values are expected and are retained. The transformation is invertible up to floating-point rounding: $U = \sigma Z + \mu$. It does not make the image distribution Gaussian or remove the background.

Table 5: Numerical scales across all training pixels. Values are rounded; standardization is global, not performed separately for each image.

Representation	Mean	Population SD	Minimum	Maximum
Raw intensity	72.983297	90.055735	0	255
Scaled intensity	0.286209	0.353160	0	1
Standardized intensity	≈ 0	≈ 1	-0.810424	2.021156

2.4.3 Preventing Leakage Between Partitions

The split described in Section 2.2 is fixed before fitting the normalization. Only its 54,000 training images contribute to μ and σ. The validation and test transforms reuse those constants without recalculation; their own normalized distributions are not required to have exactly zero mean or unit standard deviation. Fitting statistics on all 70,000 images, or normalizing each evaluation partition using its own statistics, would allow held-out information to influence the input transformation.

The cached training and validation indices, normalization constants, source fingerprint, and split fingerprint identify the preprocessing used by a run. All five architectures use this same split and fitted numerical transform. Validation loss selects the checkpoint, while the official test set evaluates the selected weights. Neither test labels nor test performance determine the normalization or the choice of representation in the baseline comparison.

2.4.4 Input Representations for the Five Models

After standardization, the representation utility arranges the same 784 values for each architecture. Table 6 gives the shapes after batching, where B is the number of images in a batch. The normal batch size is 128; the final batch may contain fewer images. Inputs remain `float32`. Each image keeps one integer target in $\{0, \ldots, 9\}$, giving a target tensor of shape $(B,)$. Labels are not standardized or converted into image features.

Table 6: Model input layouts after common standardization. Shapes precede learned feature extraction or patch embedding. Each image retains 784 values.

Model	Batch shape	Ordering and interpretation
Linear / MLP	$(B, 784)$	One vector per image, flattened row by row.
CNN	$(B, 1, 28, 28)$	One grayscale channel with the original image grid.
LSTM	$(B, 28, 28)$	28 row steps, with 28 pixel features at each step.
Transformer	$(B, 49, 16)$	49 nonoverlapping 4×4 patches, with 16 pixel features per patch.

Linear Classifier and MLP. The image is flattened in row-major order:

$$v_{i,28r+c} = Z_{i,0,r,c}, \qquad v_i \in \mathbb{R}^{784}. \tag{7}$$

Thus, positions 0–27 contain the first row and positions 28–55 contain the second. Flattening preserves the values and their fixed positions, but the model interface does not explicitly encode a two-dimensional neighborhood or impose local weight sharing. The MLP's hidden transformations belong to the model architecture, rather than to preprocessing.

Convolutional Neural Network. The CNN receives Z_i directly as a $(1, 28, 28)$ image. Keeping the channel and spatial axes allows convolutional layers to operate on local neighborhoods. No handcrafted edge map, threshold mask, or class prototype is supplied as an additional channel.

LSTM. The single channel dimension is removed, giving

$$S_{i,t,j} = Z_{i,0,t,j}, \qquad t, j \in \{0, \ldots, 27\}. \tag{8}$$

The sequence moves from the top row to the bottom row, and each step contains the 28 left-to-right pixel values in that row. These are spatial steps within one static image. Because both axes have length 28, an accidental transpose would preserve the shape while changing the sequence; ordering must therefore be checked by comparing the actual elements.

Transformer. The image is divided into a 7×7 grid of nonoverlapping 4×4 patches. Patches are read left to right and then top to bottom, and the pixels inside each patch are also flattened row by row. If $a, b \in \{0, \ldots, 6\}$ index the patch grid and $u, v \in \{0, \ldots, 3\}$ index a pixel within a patch, then

$$P_{i,7a+b,\,4u+v} = Z_{i,0,\,4a+u,\,4b+v}. \tag{9}$$

There are 49 patches with 16 values each, so $49 \times 16 = 784$. The patch layout requires no padding or discarded boundary pixels. Projection from 16 pixel values to an embedding, positional encoding, and attention are model operations to be explained in the methodology section.

Figure 8 illustrates the layouts using one training image. Reordering preserves the pixel information; differences in how the architectures learn from these layouts will be examined experimentally.

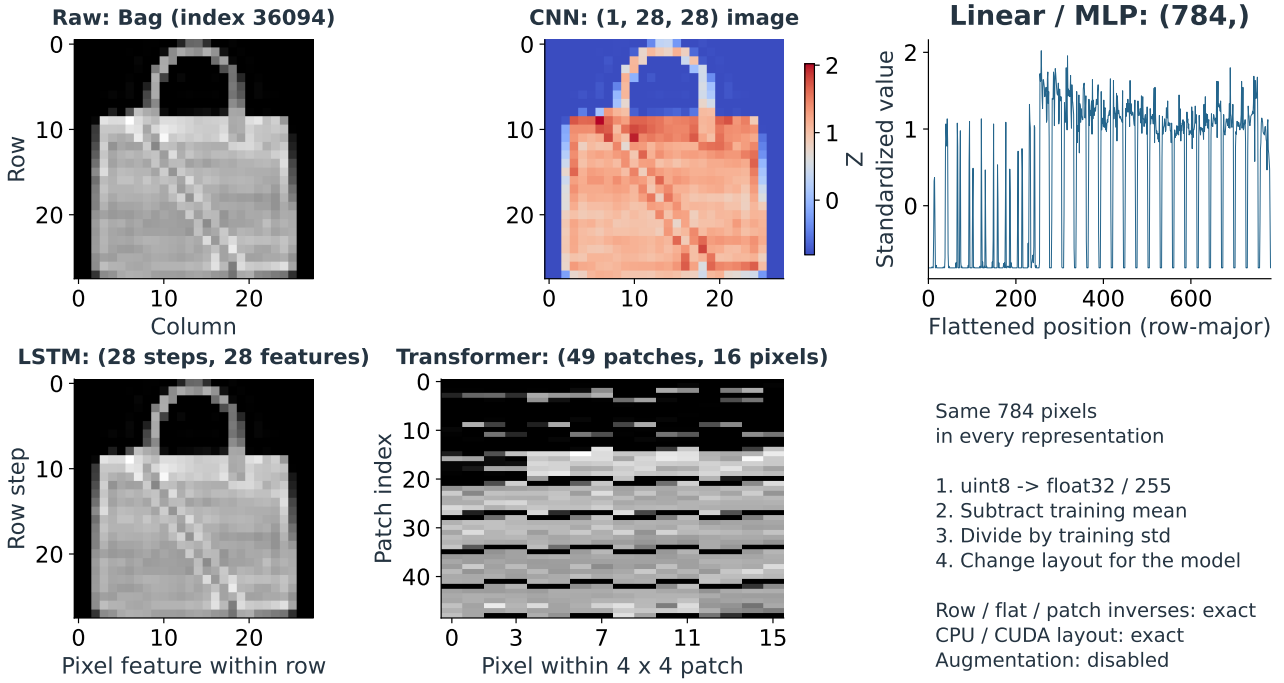


Figure 8: A training image shown as raw pixels and as standardized inputs for the five models. Vector, row, and patch layouts contain the same 784 standardized values; only their arrangement changes. The example is not a model prediction.

2.4.5 Data Retention and Augmentation Policy

The baseline retains every image and all pixel coordinates. The audit found no entirely blank training images or exact duplicates across the checked partitions. Brightness and nonzero coverage vary within each class, but extreme values alone do not establish corruption or an incorrect label. Accordingly, the EDA does not justify removing dark images, cropping stable background locations, or relabeling visually ambiguous garments.

No resizing, denoising, binarization, histogram equalization, or dimensionality reduction is applied to the model inputs. In particular, the nonzero masks, class means, and similarity matrix in the EDA remain descriptive analyses; they are not extra training features or replacements for the original images. All five models therefore start from the same grayscale information.

Random augmentation is disabled in the shared baseline configuration. This keeps the initial comparison independent of architecture-specific augmentation choices. The loader supports optional horizontal flips with probability 0.5 and rotations of up to ten degrees for training images, but these options are not part of the reported preprocessing baseline. If studied later, they must be identified as separate experiments, applied only to training inputs before standardization, and evaluated with the same deterministic validation and test transforms. No augmentation gain is assumed here.

2.4.6 Preprocessing Verification

The saved EDA audit checks the transformation against the existing data utility. Over all standardized training pixels, the measured mean is within 10^{-6} of zero and the population standard deviation is within 10^{-6} of one. Independent NumPy calculations reproduce the raw moments and pixel histogram, and the fitted normalization agrees with the split cache. The same CPU tensor conversion is used by the loader and the EDA before CUDA calculations, avoiding a change in conversion order between them.

Representation checks compare flattened and row-sequence elements with their source pixels, reconstruct the image from the patch sequence, and confirm agreement between CPU and CUDA layout conversions. These checks verify values and ordering, rather than relying only on tensor dimensions. Together with the fixed split, they establish the input interface for the methodology and implementation sections. They do not establish classifier accuracy, which requires evaluation of trained models.

2.5 Key Findings and Implications for Methodology

The dataset analysis establishes a controlled starting point for the five-model comparison. Its main findings translate into the following experimental decisions.

1. **Class balance supports an unweighted baseline.** Each class contributes 5,400 training, 600 validation, and 1,000 test images. No resampling or class weighting is introduced to address frequency imbalance. Equal counts do not guarantee equal recognition difficulty, so evaluation must include per-class precision, recall, and F1 alongside accuracy and macro-F1.

2. **Background prevalence must be separated from image validity.** Exactly 50.21% of training pixels are zero, but no training image is entirely blank. Brightness and coverage distributions overlap across classes. The baseline therefore keeps the grayscale intensities and every image rather than applying an unsupported brightness threshold or background removal rule.

3. **Spatial structure motivates different model representations.** Class means retain broad silhouettes, while variability maps and individual examples show changes that averaging conceals. Vectors provide the Linear and MLP baselines; the CNN retains the image grid; the LSTM reads ordered rows; and the Transformer receives ordered patches. All layouts retain the same 784 standardized values. Architecture comparisons must still account for differences in parameter count and computation before attributing a performance gap to representation alone.

4. **Prototype similarity gives a specific error-analysis question.** Coat and Shirt have the highest off-diagonal prototype cosine similarity, approximately 0.9873. Later confusion matrices should check whether these and other upper-body categories are frequent directional errors, and example galleries should examine the corresponding mistakes. Similarity of averages alone does not predict which model will perform best or establish that any supplied label is incorrect.

5. **One fitted preprocessing pipeline supports a consistent comparison.** The seed-42 split, cached indices, training-only normalization, and verified input ordering are shared by all architectures. Validation loss selects weights, and the selected checkpoint is evaluated on the official test set. Any change to augmentation or representation must be recorded as a separate experimental condition rather than silently changing the baseline.

The analysis has limits. Exact-image hashing cannot exclude near duplicates or related products, and a few sampled images cannot establish the frequency of an appearance. Class averages hide individual variation. The present evidence also does not measure training variability or generalization to high-resolution, colored, or unconstrained product photographs. Those limits remain relevant when interpreting later results.

Section 3 can now define the five architectures, their mathematical operations, loss, training strategy, and evaluation metrics using the input shapes established in Section 2.4. Section 4 can describe how the shared dataset views, batching, random-state control, device transfer, and checkpoint handling implement that protocol. Training histories, predictive results, timing measurements, and error analysis belong to the later experimental sections; the EDA provides hypotheses and preprocessing decisions for those experiments rather than substituting for them.